DATA ANALYTICS LAB SHEET 8

Name: ANURAG PARMAR
Roll No.: 8
Section: B
CU ID: CU24250076
B.Tech CSE

Experiment No. 8

Customer Churn Prediction using Decision Tree Classification

AIM

To build and evaluate a Decision Tree Classification model for predicting customer churn using a customer dataset and analyze factors influencing customer retention.

PURPOSE

Customer churn prediction is a supervised machine learning application used to identify customers who may discontinue a service or product. The analysis can help organizations understand retention-related factors and plan targeted retention strategies.

NOTE

This notebook uses a self-contained customer churn dataset generated inside Colab. It does not depend on an external CSV URL, so it can run without a dataset download or GitHub 404 error.

1. Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2. Create Customer Churn Dataset

```
# Self-contained synthetic customer churn dataset.
# No external internet connection is required.

rng = np.random.default_rng(42)
n = 1000

age = rng.integers(18, 76, n)
tenure = rng.integers(1, 73, n)
monthly_charges = np.round(rng.uniform(20, 130, n), 2)
support_calls = rng.poisson(2.2, n).clip(0, 10)
contract_months = rng.choice([1, 12, 24], size=n, p=[0.50, 0.30, 0.20])
late_payments = rng.poisson(1.0, n).clip(0, 6)

# Generate a realistic binary churn outcome from customer characteristics.
risk = (
    1.6
```

```
    + 0.035 * (monthly_charges - 70)
    + 0.30 * support_calls
    + 0.40 * late_payments
    - 0.025 * tenure
    - 0.035 * contract_months
    + 0.005 * (age - 45)
)

probability = 1 / (1 + np.exp(-risk))
churn = rng.binomial(1, np.clip(probability, 0.03, 0.95))

df = pd.DataFrame({
    "Age": age,
    "Tenure": tenure,
    "MonthlyCharges": monthly_charges,
    "SupportCalls": support_calls,
    "ContractMonths": contract_months,
    "LatePayments": late_payments,
    "Churn": churn
})

df["Churn"] = df["Churn"].map({0: "No", 1: "Yes"})

print("Dataset created successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset created successfully.
Dataset shape: (1000, 7)
```

	Age	Tenure	MonthlyCharges	SupportCalls	ContractMonths	LatePayments	Churn
0	23	59	26.83	4	24	2	Yes
1	62	54	70.41	2	24	4	No
2	55	41	34.19	1	1	0	No
3	43	64	36.76	0	24	1	No
4	43	10	89.55	1	1	1	Yes

3. Data Inspection and Preprocessing

```
print("Data types:")
print(df.dtypes)

print("\nMissing values:")
display(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nTarget distribution:")
display(df["Churn"].value_counts())

# Remove duplicate records if present.
df = df.drop_duplicates().copy()

print("\nFinal dataset shape:", df.shape)
```

```
Data types:
Age                int64
Tenure             int64
MonthlyCharges   float64
SupportCalls       int64
ContractMonths     int64
LatePayments       int64
Churn             object
dtype: object

Missing values:
```

	0
Age	0
Tenure	0
MonthlyCharges	0
SupportCalls	0
ContractMonths	0
LatePayments	0
Churn	0

dtype: int64

```
Duplicate rows: 0

Target distribution:
```

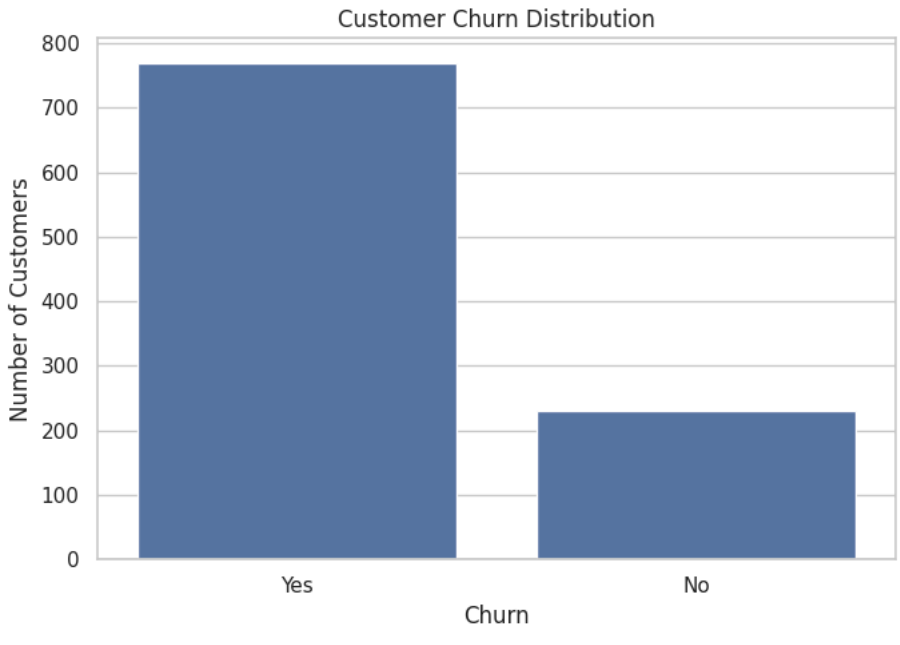
Churn	count
Yes	769
No	231

dtype: int64

```
Final dataset shape: (1000, 7)
```

4. Exploratory Data Analysis

```
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Churn")
plt.title("Customer Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

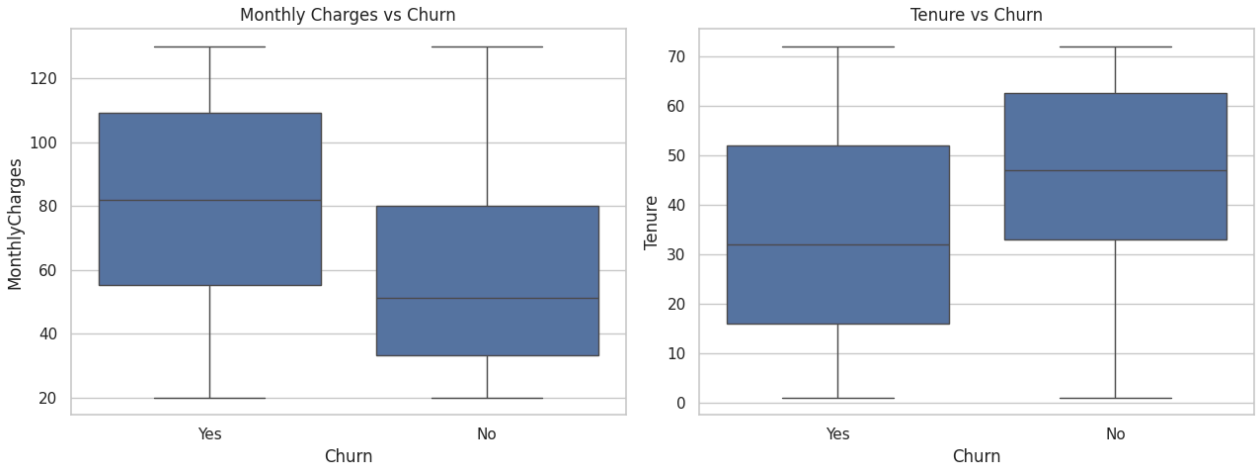


```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges", ax=axes[0])
axes[0].set_title("Monthly Charges vs Churn")

sns.boxplot(data=df, x="Churn", y="Tenure", ax=axes[1])
axes[1].set_title("Tenure vs Churn")

plt.tight_layout()
plt.show()
```



5. Prepare Features and Target

```
X = df.drop(columns=["Churn"])
y = df["Churn"].map({"No": 0, "Yes": 1})

print("Features:")
print(X.columns.tolist())

print("\nTarget:")
print("Churn (0 = No, 1 = Yes)")
```

```
Features:
['Age', 'Tenure', 'MonthlyCharges', 'SupportCalls', 'ContractMonths', 'LatePayments']

Target:
Churn (0 = No, 1 = Yes)
```

6. Split Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Train-test ratio: 80:20")
```

```
Training samples: 800
Testing samples: 200
Train-test ratio: 80:20
```

7. Train Decision Tree Classifier

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)

print("Decision Tree Classifier trained successfully.")
print("Criterion: Gini Index")
print("Maximum tree depth: 5")
```

```
Decision Tree Classifier trained successfully.
Criterion: Gini Index
Maximum tree depth: 5
```

8. Predict Customer Churn

```
y_pred = model.predict(X_test)

prediction_table = pd.DataFrame({
    "Actual Churn": y_test.values,
    "Predicted Churn": y_pred
})

display(prediction_table.head(15))
```

	Actual Churn	Predicted Churn
0	0	1
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1
6	1	1
7	0	1
8	1	0
9	0	1
10	1	1
11	1	1
12	0	0
13	1	1
14	1	1

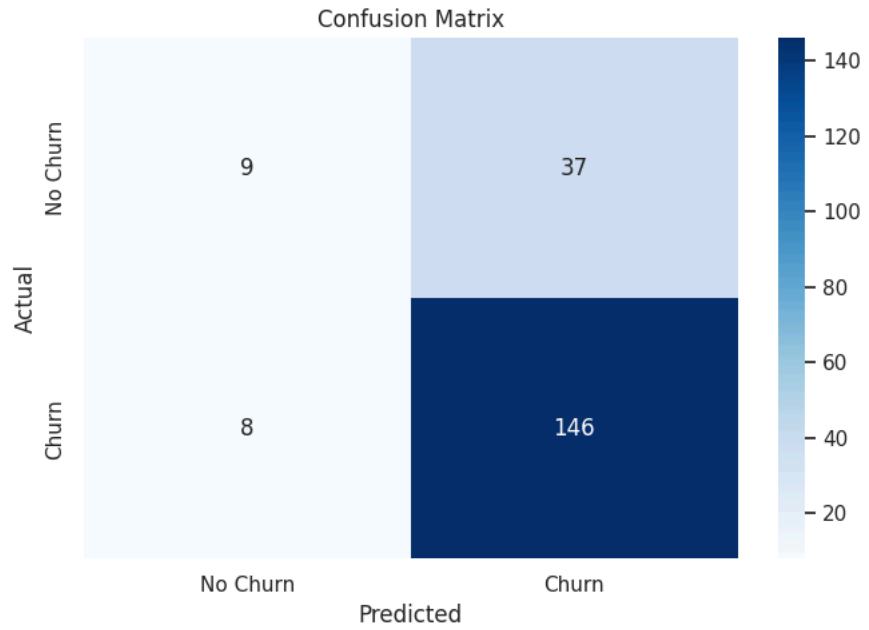
9. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```

```
print("Confusion Matrix:")
print(cm)
```




```
Confusion Matrix:
[[  9  37]
 [  8 146]]
```

10. Model Evaluation

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

metrics = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Value": [accuracy, precision, recall, f1]
})

display(metrics)

print(classification_report(
    y_test,
    y_pred,
    target_names=["No Churn", "Churn"],
    zero_division=0
))
```

	Metric	Value
0	Accuracy	0.775000
1	Precision	0.797814
2	Recall	0.948052
3	F1-Score	0.866469

```
              precision    recall  f1-score   support

    No Churn       0.53      0.20      0.29        46
       Churn       0.80      0.95      0.87       154

    accuracy                           0.78       200
   macro avg       0.66      0.57      0.58       200
weighted avg       0.74      0.78      0.73       200
```

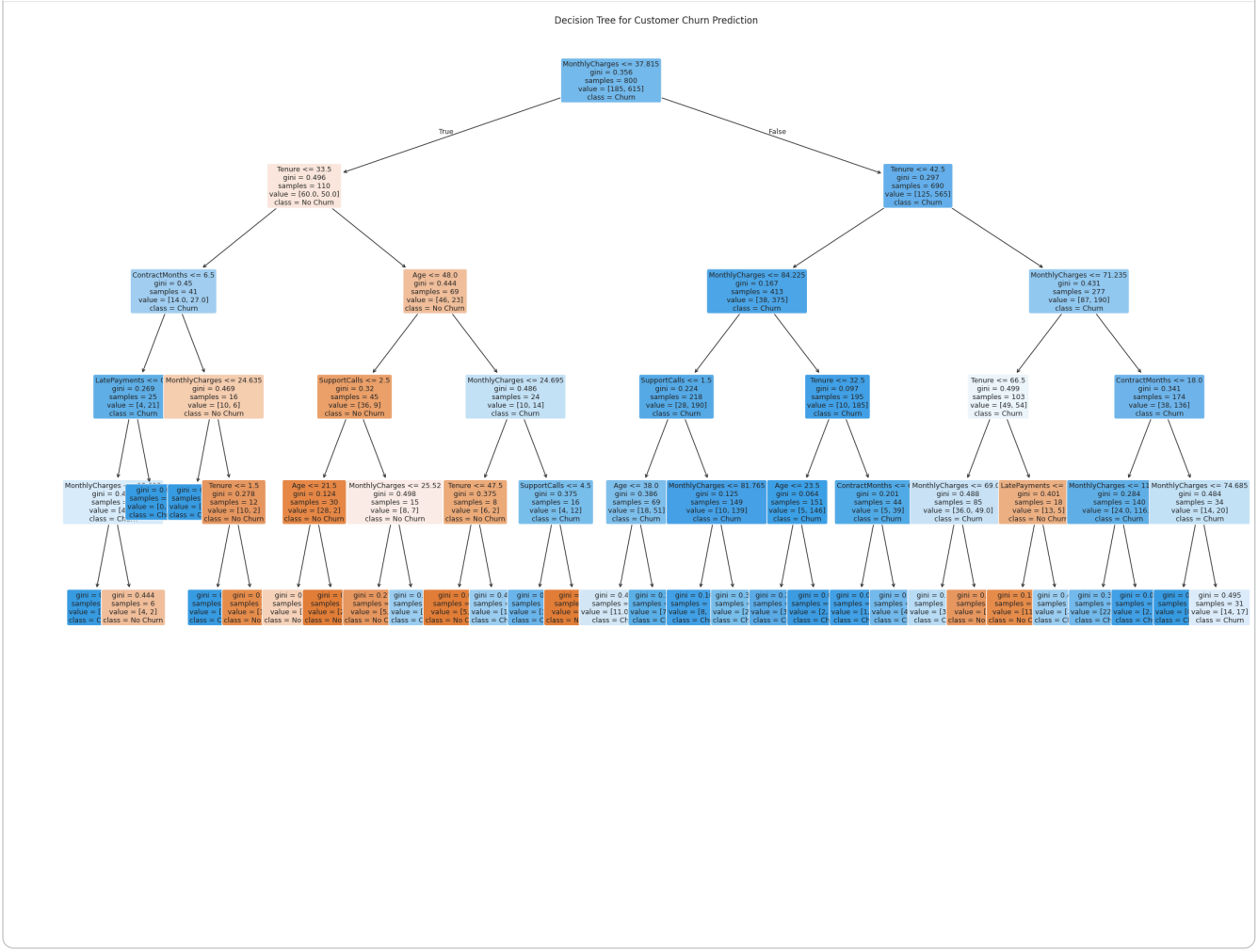
Next steps: Generate code with `metrics` | New interactive sheet

11. Visualize the Decision Tree

```
plt.figure(figsize=(22, 12))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True,
    fontsize=9
)

plt.title("Decision Tree for Customer Churn Prediction")
plt.tight_layout()
plt.show()
```



12. Feature Importance

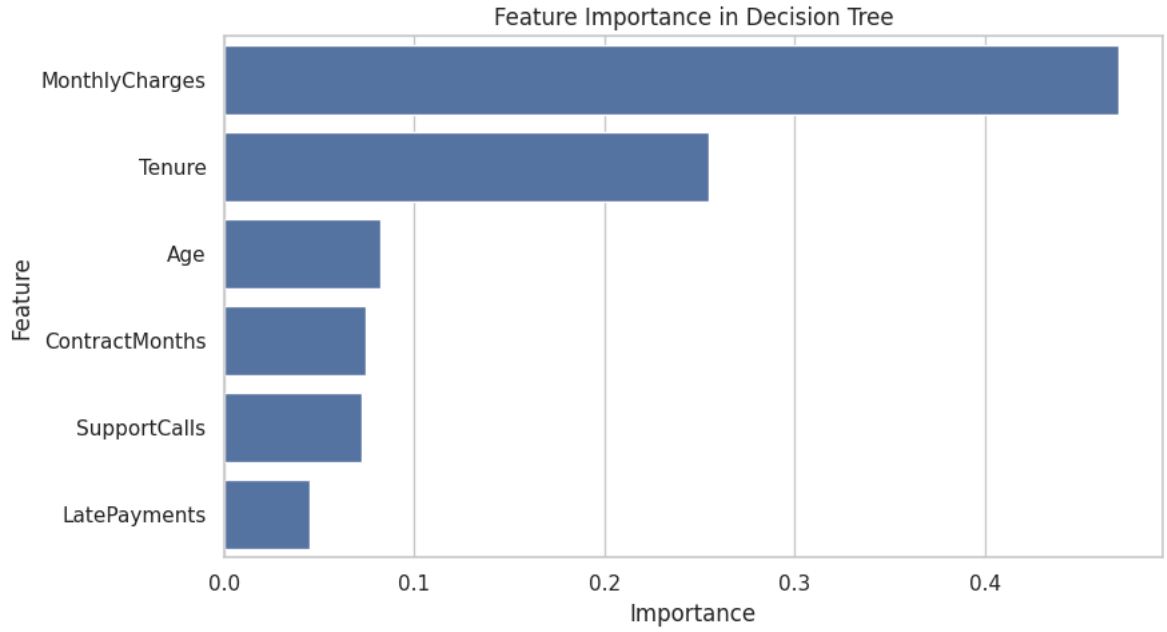
```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values("Importance", ascending=False)

display(importance)

plt.figure(figsize=(9, 5))
sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance in Decision Tree")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

	Feature	Importance
2	MonthlyCharges	0.470055
1	Tenure	0.255195
0	Age	0.082553
4	ContractMonths	0.074552
3	SupportCalls	0.072203
5	LatePayments	0.045442



Next steps: Generate code with importance | New interactive sheet

13. Analyze Important Churn Factors

```
top_features = importance.head(3)

print("Top three features according to Decision Tree importance:")
for i, row in enumerate(top_features.itertuples(index=False), start=1):
    print(f"{i}. {row.Feature}: {row.Importance:.4f}")

print("\nInterpretation:")
print("Features with larger importance values contributed more to the decision-tree predictions in this dataset.")
print("These model-derived relationships should be interpreted as predictive associations rather than proof of causation.")
```

```
Top three features according to Decision Tree importance:
1. MonthlyCharges: 0.4701
2. Tenure: 0.2552
3. Age: 0.0826

Interpretation:
Features with larger importance values contributed more to the decision-tree predictions in this dataset.
These model-derived relationships should be interpreted as predictive associations rather than proof of causation.
```

14. Business Recommendations

```
print("BUSINESS RECOMMENDATIONS")
print("------------------------")
print("1. Identify customers predicted to have higher churn risk and prioritize them for retention outreach.")
print("2. Review customers with repeated support calls and investigate unresolved service issues.")
print("3. Monitor customers with high monthly charges and evaluate suitable plans or targeted offers.")
print("4. Encourage longer-term contracts where appropriate through transparent loyalty benefits.")
print("5. Monitor payment behavior and provide suitable reminders or support for repeated late payments.")
print("6. Use the model as a decision-support tool and combine predictions with business context before taking action.")
```

```
BUSINESS RECOMMENDATIONS
------------------------
1. Identify customers predicted to have higher churn risk and prioritize them for retention outreach.
2. Review customers with repeated support calls and investigate unresolved service issues.
```

```
3. Monitor customers with high monthly charges and evaluate suitable plans or targeted offers.
4. Encourage longer-term contracts where appropriate through transparent loyalty benefits.
5. Monitor payment behavior and provide suitable reminders or support for repeated late payments.
6. Use the model as a decision-support tool and combine predictions with business context before taking action.
```

RESULT

A Decision Tree Classification model was successfully developed for customer churn prediction. The dataset was prepared, divided into training and testing sets using an 80:20 ratio, and a Decision Tree classifier was trained using the Gini splitting criterion. Churn predictions were evaluated using a Confusion Matrix, Accuracy, Precision, Recall, and F1-Score. The decision tree and feature importance were visualized to understand the model's decision process and important predictive factors.

Answers to Questions

Q1. What is classification, and how does it differ from regression?

Classification is a supervised machine learning task in which the model predicts a categorical class or label. Regression predicts a continuous numerical value. For example, churn prediction is classification, while predicting monthly sales is regression.

Q2. Explain the working principle of the Decision Tree algorithm.

A Decision Tree repeatedly divides data using feature-based rules. At each node, a split is selected according to a criterion such as Gini Index or Entropy. The process continues until stopping conditions are reached, and the final leaf node provides the predicted class.

Q3. Differentiate between Gini Index and Entropy as splitting criteria.

Both measure impurity in a node. Gini Index is based on the probability of incorrect classification if an observation is randomly labeled according to the class distribution. Entropy measures the uncertainty or disorder in the node using logarithmic probabilities. Both can be used to select useful splits.

Q4. What is a Confusion Matrix? Explain its components.

A Confusion Matrix summarizes classification predictions using four components: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). These values compare predicted classes with actual classes.

Q5. Define Accuracy, Precision, Recall, and F1-Score. Why are these metrics important?

Accuracy is the proportion of all correct predictions. Precision is the proportion of predicted positives that are actually positive. Recall is the proportion of actual positives correctly identified. F1-Score is the harmonic mean of Precision and Recall. These metrics provide different views of classification performance and are useful when class errors have different consequences.

Q6. What is overfitting in a Decision Tree? How can it be reduced?

Overfitting occurs when a tree becomes excessively complex and learns noise or highly specific patterns in the training data. It can be reduced using maximum depth, minimum samples per split/leaf, pruning, cross-validation, and appropriate feature selection.

Q7. Why is customer churn prediction important for businesses?

Churn prediction can help businesses identify customers who may discontinue their service. Early identification allows organizations to investigate customer issues, plan retention activities, improve customer experience, and potentially reduce revenue loss.

Q8. What are the advantages and limitations of the Decision Tree algorithm?

Advantages include easy interpretation, visual representation, handling of numerical and categorical variables after suitable preprocessing, and relatively simple prediction rules. Limitations include possible overfitting, sensitivity to small changes in data, and potentially unstable trees.

Q9. Mention three real-world applications of Decision Tree Classification other than customer churn prediction.

1. Loan approval and credit-risk classification.
2. Medical disease classification.
3. Spam email or message classification.

Q10. How can the insights obtained from a churn prediction model help organizations improve customer retention and business profitability?